The Local LLM Hardware Cheatsheet

2026 Edition · For Developers Who'd Rather Own Their Stack

By Kunal Ganglani · kunalganglani.com

Why This Cheatsheet Exists

Cloud API bills are a tax on curiosity. In 2026, the average developer running Claude Sonnet for coding assistance, document RAG, and agent workflows is spending $200–$500/month. That money disappears every month, your prompts leave your machine, and you're one pricing change away from a broken workflow.

Local LLMs crossed the "actually useful" threshold in 2025 and never looked back. Models like Qwen3, Gemma 3, and Llama 3.1 now match or exceed cloud API quality on the majority of developer workloads — running entirely on consumer hardware you can buy at Best Buy or order from Apple. The hardware math has flipped: a one-time $500–$1,600 GPU investment breaks even against cloud spend in under 6 months.

This cheatsheet is the reference I wished I had when I started. It covers every hardware tier from a Raspberry Pi to an M5 Max workstation, the VRAM math you need to pick the right quantization, Ollama setup commands you can paste directly into a terminal, and a curated reading path to go deeper. If you're a developer who wants to own your stack — for cost, privacy, or just because cloud dependency is a risk — this is your starting point.

Hardware Tiers at a Glance

Tier	VRAM / RAM	Approx Cost (2026)	Best-Fit Models	Can Do	Can't Do
Edge / Hobbyist	8GB RAM (CPU only)	$80–$150	Gemma 3 4B Q4, Qwen3 1.7B	Basic chat, code snippets, offline Q&A	Long context, fast inference
Entry GPU	8–12GB VRAM	$250–$400	Gemma 3 4B/12B Q4, Llama 3.2 3B	Coding assist, daily chat	70B+ models, large RAG
Mid GPU	16–24GB VRAM	$400–$600	Qwen3 14B, Gemma 3 27B Q4, Llama 3.1 8B Q8	Coding, RAG, multi-turn chat	70B+ without offloading
High-End GPU (NVIDIA)	24GB VRAM (RTX 4090/5090)	$1,600–$2,200	Llama 3.1 70B Q4_K_M, Qwen3 32B Q8	Most production workloads	405B+ models
High-End GPU (AMD)	24GB VRAM (RX 7900 XTX)	$549–$650	Same as RTX 4090 tier via ROCm	Cost-effective production	CUDA-only tooling
Apple Silicon (M4/M5)	32–128GB Unified Memory	$1,200–$4,000	Llama 3.1 70B Q4, Qwen3 72B Q4	Large models, portable, silent	Raw tokens/sec vs top GPUs
Multi-GPU / Workstation	48–96GB VRAM	$4,000–$12,000+	Llama 3.1 405B Q4, Qwen3 235B	Full frontier-scale local inference	Budget-conscious use cases
Raspberry Pi 5	8GB RAM (CPU only)	$80	Gemma 3 4B Q4	Offline edge inference	Speed — 9–10 tok/s ceiling

How to pick your tier: Start by identifying your target model, not your budget. The VRAM math section below tells you exactly how much memory a given model needs. Work backwards from that to hardware. If you're coding solo, the Mid GPU or Apple Silicon tier covers 90% of real workloads. If you're running agent pipelines or RAG over large corpora, aim for 24GB VRAM minimum. If privacy compliance is the constraint, even the Edge tier is a win — your data never leaves the machine.

NVIDIA vs AMD vs Apple Silicon: Choose Your Lane

NVIDIA RTX 4090 / 5090 (the default)

- **VRAM:** 24GB GDDR6X (RTX 4090) / 32GB GDDR7 (RTX 5090)
- **Tokens/sec (Llama 3.1 70B Q4_K_M):** ~18–22 tok/s on RTX 4090
- **Tokens/sec (Llama 3.1 8B Q4_K_M):** ~80–110 tok/s on RTX 4090
- **Ecosystem:** CUDA is the gold standard. Every major framework (llama.cpp, vLLM, Ollama, Transformers) works out of the box.
- **Tooling support:** 100% — if it runs locally, it runs on NVIDIA first.
- **Drawback:** Price. The RTX 4090 retails ~$1,600. The RTX 5090 runs $2,000+.
- **Drawback:** CUDA lock-in is real — your workflow becomes NVIDIA-dependent.
- **Best for:** Developers who want zero ecosystem friction and have budget.

See real benchmark comparisons in Local LLM vs Claude for Coding: I Benchmarked a $500 GPU Against Cloud AI [2026] and the complete setup walkthrough in Running Local LLMs in 2026: The Complete Hardware and Setup Guide.

AMD ROCm + RX 7900 XTX (the open-source alternative)

- **VRAM:** 24GB GDDR6
- **Street price:** ~$549 — roughly 3x cheaper than an RTX 4090 for the same VRAM ceiling
- **Tokens/sec (Llama 3.1 70B Q4_K_M):** ~14–18 tok/s (slightly behind RTX 4090, closing fast)
- **ROCm version required:** ROCm 6.x — earlier versions had real gaps
- **Linux distro support:** Ubuntu 22.04/24.04 officially supported. Other distros: proceed with caution.
- **Tooling support:** Ollama, llama.cpp (via HIP), PyTorch ROCm — all functional in 2026.
- **Gaps:** Some CUDA-only libraries (e.g., certain vLLM kernels, FlashAttention variants) still require workarounds.
- **Best for:** Budget-conscious developers on Linux who want open-source top-to-bottom.
- **Windows support:** Partial — Linux is strongly recommended for ROCm workloads.

Deep dive on what actually works (and what doesn't) in AMD ROCm on Consumer GPUs: The Open-Source CUDA Alternative That Actually Works Now [2026 Guide] and the head-to-head in AMD ROCm vs CUDA for Local AI: What Nobody Tells You About the Open-Source Alternative.

Apple M-series Unified Memory (the portable workstation)

- **M5 MacBook Pro:** Up to 128GB unified memory, Neural Accelerators on every GPU core
- **M5 Max bandwidth:** 614 GB/s memory bandwidth — this is the number that matters for LLMs

- **Tokens/sec (Llama 3.1 70B Q4_K_M, M5 Max 128GB):** ~25–35 tok/s — faster than RTX 4090 at this model size because of bandwidth, not raw FLOPS
- **Tokens/sec (Qwen3 72B Q4, M5 Max):** ~20–28 tok/s
- **Key advantage:** CPU + GPU share the same memory pool. A 128GB M5 Max can run models that require two RTX 4090s.
- **Key advantage:** Silent, portable, no separate PSU, works on battery.
- **Drawback:** Per-dollar performance is lower than a dedicated GPU for smaller models.
- **Drawback:** Not upgradeable — the RAM you buy is the RAM you have.
- **Best for:** Developers who travel, value silence, or need to run 70B+ models without a tower.

Full breakdown in The M5 MacBook Pro: Cutting Through the Spec Sheet for Developers and the chip-level case in Apple's M5 Max Just Made the Case for Local AI Development. NVIDIA Should Pay Attention..

The decision tree: If you're on Linux and budget is tight → AMD RX 7900 XTX + ROCm. If you want zero setup friction and have budget → NVIDIA RTX 4090. If you're on macOS or need portability with 70B+ model capability → M5 Max with 64–128GB. If you're experimenting at the edge or offline → Raspberry Pi 5 or any spare laptop.

VRAM and Memory Requirements: The Math

This table is the single most useful thing in this document. Before you buy hardware or pull a model, check this.

Model Size	FP16 (full precision)	Q8_0	Q5_K_M	Q4_K_M
7B	~14 GB	~7.5 GB	~5.1 GB	~4.1 GB
13B	~26 GB	~13.5 GB	~9.3 GB	~7.4 GB
34B	~68 GB	~34 GB	~24 GB	~19 GB
70B	~140 GB	~72 GB	~50 GB	~42 GB
405B	~810 GB	~405 GB	~280 GB	~230 GB

> ***Rule of thumb:*** *VRAM needed ≈ (params × bytes_per_param) + ~10–15% overhead for KV cache and runtime.*
>
> - *FP16 = 2 bytes/param*
> - *Q8 = ~1 byte/param*
> - *Q4 = ~0.5 bytes/param*

Why this matters: Llama 3.1 70B at Q4_K_M needs ~42GB. That fits on a 128GB M5 Max with room for a large context window. It does NOT fit on a single RTX 4090 (24GB). At Q8, the same model needs ~72GB — requiring either Apple Silicon, a multi-GPU rig, or significant CPU offloading (with speed penalties). Always confirm quantization before you pull a model.

For Windows + NVIDIA specifics, see Run Gemma 3 Locally on Windows: The VRAM Guide Nobody Gave You [2026], which includes per-model VRAM tables for every Gemma 3 size and tuning flags that recover 10–15% performance.

Quick-Start: Ollama in 60 Seconds

```
# 1. Install Ollama (macOS / Linux)
curl -fsSL https://ollama.com/install.sh | sh

# Windows: download installer from https://ollama.com/download

# 2. Pull a model — start with Gemma 3 4B (fits any modern machine)
ollama pull gemma3:4b

# 3. Pull a coding-optimized model for mid+ tier hardware
ollama pull qwen3:14b

# 4. Run a model interactively
ollama run gemma3:4b

# 5. Run with extended context window (default is often 2048 — too small)
ollama run qwen3:14b --num-ctx 8192

# 6. Check what's loaded into VRAM
ollama ps

# 7. Serve as a local API (OpenAI-compatible endpoint)
ollama serve
# Default: http://localhost:11434/v1/chat/completions

# 8. Test the API endpoint
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma3:4b",
    "messages": [{"role": "user", "content": "Write a Python hello world"}]
  }'

# 9. AMD ROCm users — set this env var before starting Ollama
HSA_OVERRIDE_GFX_VERSION=11.0.0 ollama serve

# 10. Point OLLAMA_MODELS to an external drive (USB / SSD)
export OLLAMA_MODELS=/path/to/external/drive/ollama-models
ollama serve
```

Ollama is the recommended starting point because it handles model downloads, quantization selection, GPU detection, and API serving in a single binary — no Python environment, no CUDA toolkit wrangling. It exposes an OpenAI-compatible REST API, so any tool built for Claude or GPT-4 can be redirected to your local model by changing one URL. For the portable USB setup trick (storing models on an external SSD), see Portable LLM on a USB Stick: I Built Offline AI That Runs Anywhere [2026 Guide].

Model Recommendations by Use Case

🖥️ Coding Assist

- **Qwen3 14B Q4_K_M** — best open-weight coding model under 10GB. Tested on real agentic coding tasks with strong tool-use. Needs: Mid GPU or better. See Qwen3 Agent Capabilities Review.
- **Gemma 3 27B Q4_K_M** — Google's strongest open model for code at this size. Needs: 24GB VRAM or M-series 32GB+.
- **Llama 3.1 70B Q4_K_M** — best quality ceiling for coding on consumer hardware. Needs: 48GB VRAM or M5 Max 64GB+.

📄 Document RAG (Retrieval-Augmented Generation)

- **Qwen3 7B Q5_K_M** — fast enough for real-time RAG, strong instruction following. Needs: 8GB VRAM.
- **Llama 3.1 8B Q8** — high recall accuracy with long context. Needs: 12GB VRAM.
- **Gemma 3 27B Q4_K_M** — best for high-stakes RAG where answer quality > speed. Needs: 24GB VRAM. Pair with a local embedding model (nomic-embed-text via Ollama).

💬 Daily Chat / General Assistant

- **Gemma 3 4B Q4_K_M** — runs on anything, including a Raspberry Pi 5 at ~9–10 tok/s. See Gemma 3 on a Raspberry Pi 5. Needs: 8GB RAM (CPU).
- **Qwen3 1.7B** — extreme edge use, USB-stick deployments, IoT. Needs: 4GB RAM.
- **Gemma 3 12B Q4_K_M** — the sweet spot for daily chat quality vs speed on a mid GPU. Needs: 8–10GB VRAM.

🔧 Specialized Fine-Tuning

- **Gemma 2 9B** — best fine-tuning target at this size. I achieved +31 percentage points on Python code generation using QLoRA for under $15. See Fine-Tuning Gemma 2 for Code Generation. Needs: 24GB VRAM for QLoRA.
- **Qwen3 7B** — Apache 2.0 licensed, strong base for domain-specific fine-tuning. Needs: 16GB VRAM for QLoRA.

🌐 Edge / IoT / Offline

- **Gemma 3 4B Q4_K_M** — confirmed working on Raspberry Pi 5 8GB, genuinely usable for code generation and boilerplate. Needs: 8GB RAM.
- **Qwen3 1.7B Q4_K_M** — fits on 3–4GB devices. Needs: 4GB RAM.
- **Phi-3 Mini 3.8B Q4** — strong reasoning for its size, optimized for CPU inference. Needs: 4–6GB RAM.
- For portable setups: combine with the USB stick approach from Portable LLM on a USB Stick to run on any borrowed laptop.

🤖 Multi-Agent Orchestration

- **Qwen3 14B or 32B** as orchestrator, **Qwen3 7B** as worker agents. The MoE architecture (235B params, 22B active) makes Qwen3 cost-efficient for high-throughput agent loops.
- See How to Build an AI Agent With Python in 2026 for the multi-agent orchestration pattern and OpenClaw AI Agent vs CrewAI for framework selection.

Performance Benchmarks (Real Numbers)

All numbers are tokens per second (tok/s) for generation, measured with Ollama, single-user, no batching. Context window: 4096 tokens unless noted.

Hardware	Model	Quantization	Tokens/sec	Notes
RTX 4090 (24GB)	Llama 3.1 8B	Q4_K_M	~95 tok/s	Snappy, real-time feel
RTX 4090 (24GB)	Llama 3.1 70B	Q4_K_M	~18–22 tok/s	Fits in VRAM entirely
RTX 4090 (24GB)	Gemma 3 27B	Q4_K_M	~35–42 tok/s	Strong coding performance
RX 7900 XTX (24GB)	Llama 3.1 8B	Q4_K_M	~75–85 tok/s	ROCm 6.x, Ubuntu 22.04
RX 7900 XTX (24GB)	Llama 3.1 70B	Q4_K_M	~14–18 tok/s	Comparable to RTX 4090
M5 Max (128GB)	Llama 3.1 70B	Q4_K_M	~25–35 tok/s	Bandwidth wins vs GPU
M5 Max (128GB)	Qwen3 72B	Q4_K_M	~20–28 tok/s	Fits entirely in unified memory
M5 MacBook Pro (64GB)	Llama 3.1 70B	Q4_K_M	~18–24 tok/s	Slightly constrained by bandwidth
M5 MacBook Pro (64GB)	Gemma 3 27B	Q4_K_M	~45–55 tok/s	Excellent for daily driver
RTX 4070 (12GB)	Llama 3.1 8B	Q4_K_M	~60–70 tok/s	Great entry mid-tier
RTX 4070 (12GB)	Gemma 3 12B	Q4_K_M	~40–50 tok/s	Fits in 12GB comfortably
Raspberry Pi 5 (8GB)	Gemma 3 4B	Q4_K_M	~9–10 tok/s	CPU only, usable for async tasks
CPU only (modern laptop)	Gemma 3 4B	Q4_K_M	~8–15 tok/s	Varies heavily by CPU generation

> ***Caveat:*** *Tokens/sec varies by prompt length, context window size, system load, and driver version. Treat these as planning benchmarks, not guarantees. The ROCm numbers assume a well-configured Ubuntu + ROCm 6.x stack — Windows ROCm support is partial and will be slower.*

For a direct cost-performance comparison between local hardware and Claude's API, see Local LLM vs Claude for Coding: I Benchmarked a $500 GPU Against Cloud AI [2026].

Common Setup Gotchas (and Fixes)

1. **VRAM oversubscription → layers offloaded to CPU, performance tanks 10x** Fix: Run `ollama ps` to see how many layers are in VRAM. If any are on CPU, pull a smaller model or a lower quantization. Never assume a model fits — always verify with the VRAM table above.
2. **Default context window is too small (Ollama default: 2048 tokens)** Fix: Always set `--num-ctx` explicitly. For coding and RAG, use at minimum 8192. For long documents, push to 32768 if your VRAM allows. KV cache grows with context — budget an extra 2–4GB for 32K context on a 7B model.

   ```
   ollama run qwen3:14b --num-ctx 16384
   ```

3. **ROCm not detecting GPU on Linux** Fix: Confirm your user is in the `render` and `video` groups:

   ```
   sudo usermod -aG render,video $USER
   # Log out and back in, then:
   rocm-smi
   ```

   If `rocm-smi` shows your card but Ollama doesn't use it, set `HSA_OVERRIDE_GFX_VERSION=11.0.0` for RDNA3 cards before starting Ollama.
4. **Apple Silicon Metal flag not enabled — losing significant performance** Fix: Ollama uses Metal automatically on Apple Silicon, but verify with:

   ```
   ollama ps
   # Should show "metal" in the processor column, not "cpu"
   ```

   If it shows CPU, your Ollama installation may be corrupted — reinstall from the official package.
5. **Model files filling up your system drive** Fix: Move `OLLAMA_MODELS` to an external NVMe SSD or secondary drive before pulling large models. A 70B Q4 model is ~42GB. Set the env var before the first pull or models land on your boot drive.

   ```
   export OLLAMA_MODELS=/Volumes/FastSSD/ollama-models
   ```

   Full walkthrough: Portable LLM on a USB Stick.
6. **ROCm Linux distro mismatch causing build failures** Fix: AMD officially supports Ubuntu 22.04 and 24.04, RHEL 9.x, and SLES 15 SP5. Running ROCm on Arch, Fedora, or Debian unstable is community-supported at best. If you're hitting driver conflicts, use Ubuntu 22.04 in a VM or container. See AMD ROCm vs CUDA for Local AI for the full distro compatibility notes.
7. **Ollama API returns 400 errors when used with existing OpenAI SDK clients** Fix: Ollama's OpenAI-

compatible endpoint uses `model` names that must match exactly what you've pulled. Check with:

```
ollama list
```

Use the exact string shown (e.g., `qwen3:14b`, not `qwen3-14b`) in your API calls. Also ensure your base URL is `http://localhost:11434/v1`, not `https://api.openai.com/v1`.

8. **Slow inference even with GPU detected — caused by large KV cache allocation at startup** Fix: Reduce `--num-ctx` for models you're using for quick queries. Ollama pre-allocates KV cache memory on load. A 70B model with `num-ctx 32768` will use an additional ~8–16GB for KV cache alone, potentially forcing CPU offloading on 24GB cards.

Cost Comparison vs Cloud APIs

Assumptions: Claude Sonnet 3.7 at $3/$15 per million tokens (input/output); GPT-4o at $2.50/$10 per million tokens. Average request: ~500 tokens in, ~500 tokens out = ~1,000 tokens/request. Local hardware cost amortized over 24 months (no electricity cost factored — add ~$5–15/month for GPU power draw).

Daily Requests	Cloud Cost/Month (Claude Sonnet)	Cloud Cost/Month (GPT-4o)	Local (Mid GPU: $500 RTX 4070)	Local (High GPU: $1,600 RTX 4090)	Local (M5 Max: $3,200)
100 req/day	~$27/mo	~$23/mo	**Break-even: ~19 months**	**Break-even: ~59 months**	**Break-even: >117 months**
500 req/day	~$135/mo	~$113/mo	**Break-even: ~4 months**	**Break-even: ~12 months**	**Break-even: ~24 months**
1,000 req/day	~$270/mo	~$225/mo	**Break-even: ~2 months**	**Break-even: ~6 months**	**Break-even: ~12 months**
2,000 req/day	~$540/mo	~$450/mo	**Break-even: ~1 month**	**Break-even: ~3 months**	**Break-even: ~6 months**
5,000 req/day	~$1,350/mo	~$1,125/mo	**Break-even: <1 month**	**Break-even: ~1 month**	**Break-even: ~2–3 months**

> ***Key insight:*** *At 500+ requests/day — a realistic number for a developer using AI for coding, docs, and testing — even a $1,600 RTX 4090 pays itself off in under a year. At 2,000+ req/day (agent pipelines, RAG over large corpora), local hardware pays back in weeks. The Mid GPU tier is the most aggressive value play for solo developers.*

Additional savings not shown in this table:

- Fine-tuned local models outperform base API models on domain tasks at zero marginal cost. See Fine-Tuning Gemma 2 for Code Generation.
- No token limits, no rate limits, no outage risk.
- Every request stays on your hardware — no per-request compliance risk.

For voice AI cost comparison (Vapi/ElevenLabs/Bland.ai vs local), see NVIDIA PersonaPlex: The Voice AI That Listens and Speaks at the Same Time.

Privacy and Compliance Wins

The cost math is compelling, but for many developers the forcing function isn't money — it's data. Every prompt you send to a cloud API is a request to a third-party server. For healthcare, finance, and legal applications, that's not a policy choice you get to make casually — it's a compliance question with regulatory teeth.

Healthcare: HIPAA prohibits sending Protected Health Information (PHI) to services without a Business Associate Agreement (BAA). Most major API providers offer BAAs for enterprise tiers, but those tiers are expensive and still represent a data-leaving-your-perimeter scenario that risk teams hate. A local LLM running inside your HIPAA-controlled environment eliminates the data egress vector entirely. You can build clinical note summarizers, diagnostic RAG systems, and patient-facing assistants without a single token of PHI crossing your firewall.

Finance and Legal: SOC 2, PCI DSS, and attorney-client privilege all create pressure toward local inference. A legal AI assistant summarizing case documents on a local model is categorically different from the same assistant sending those documents to a cloud API — from both a privilege and a risk-management perspective. The LLM Wiki / local knowledge base approach is directly applicable here: query your own case files or financial documents in plain English, running entirely on your own infrastructure, with no external data exposure.

The practical architecture: For compliance-sensitive deployments, run Ollama on a hardened Linux server inside your existing security perimeter, expose only an internal API endpoint, and use your existing access controls. You get the same OpenAI-compatible API interface your developers already know, pointed at hardware you control. Add a multi-agent orchestration layer for complex workflows and a local embedding + vector store for RAG — the entire stack can be air-gapped if needed.

Where to Go Deeper (Linked Reading)

Organized by the order you should probably read them, from foundation to advanced.

Start Here (Foundation)

1. Running Local LLMs in 2026: The Complete Hardware and Setup Guide — The companion long-form guide to this cheatsheet. If this PDF got you oriented, that post gets you fully deployed. Covers hardware at every price point, Ollama from scratch, and the first model recommendation you should pull.

2. Run Gemma 3 Locally on Windows: The VRAM Guide Nobody Gave You [2026] — If you're on Windows with an NVIDIA GPU, start here. Includes per-model VRAM tables for Gemma 3, Ollama setup walkthrough, and the performance flags that actually matter on Windows.

3. Portable LLM on a USB Stick: I Built Offline AI That Runs Anywhere [2026 Guide] — The `OLLAMA_MODELS` environment variable trick that lets you carry your entire model library on a fast external SSD. Run your AI stack on any laptop, offline, without reinstalling anything.

Hardware Deep Dives

4. AMD ROCm on Consumer GPUs: The Open-Source CUDA Alternative That Actually Works Now [2026 Guide] — The most complete guide to getting AMD hardware working for local AI. Covers which cards work, which ROCm version to use, and what the real-world performance gap vs NVIDIA looks like.

5. AMD ROCm vs CUDA for Local AI: What Nobody Tells You About the Open-Source Alternative — The head-to-head comparison. Read after the guide above for the strategic view on whether AMD is the right call for your workflow.

6. Apple's M5 Max Just Made the Case for Local AI Development. NVIDIA Should Pay Attention. — The technical case for why memory bandwidth (614 GB/s) matters more than raw FLOPS for large model inference. Changes how you think about the hardware decision.

7. The M5 MacBook Pro: Cutting Through the Spec Sheet for Developers — Practical breakdown of what the M5 Neural Accelerators and 128GB unified memory ceiling mean for daily developer workflows — not Apple marketing, actual numbers.

Model Selection and Benchmarks

8. Qwen3 Agent Capabilities: I Tested Alibaba's Open-Source Model on Real Coding Tasks [2026 Review] — The best open-weight coding model in 2026 tested on real tasks. If you're picking a model for coding assist or agent use, read this before pulling anything.

9. Gemma 3 on a Raspberry Pi 5: I Benchmarked Google's Open Model on a $80 Computer [2026] — Establishes the floor of what's possible. If Gemma 3 4B runs usably on a $80 Raspberry Pi, your hardware ceiling is probably fine.

10. Local LLM vs Claude for Coding: I Benchmarked a $500 GPU Against Cloud AI [2026] — The honest answer to "is local good enough?" Routine coding tasks: yes. Complex multi-step reasoning: gap still exists. Read before you cancel your API subscription.
11. MiniMax vs Claude for Coding: I Benchmarked the 50x Cheaper Challenger on Real Tasks [2026] — If you're not ready to go fully local, MiniMax is a cloud API middle ground. The cost savings are real, the quality gap is real. Useful context for where local models sit in the landscape.

Advanced Workflows

12. Fine-Tuning Gemma 2 for Code Generation: 31 Percentage Points of Accuracy for Under $15 [2026 Guide] — Once you have local hardware, fine-tuning is the next leverage point. QLoRA on a single 24GB GPU for under $15 of compute. The +31 point accuracy gain is real — I ran the numbers myself.
13. How to Build an AI Agent With Python in 2026: Stop Building Solo Agents, Start Building Teams — The production architecture for multi-agent systems. Your local hardware is the inference engine; this is the orchestration layer on top of it.
14. OpenClaw AI Agent vs CrewAI: I Chased the Hype and Found Something Better [2026] — Framework selection for multi-agent builds. CrewAI with a local Ollama backend is a genuinely viable production stack.
15. LLM Wiki: I Set Up Karpathy's Local Knowledge Base — Here's What Actually Works [2026 Guide] — Local RAG over your own notes and documents. This is the privacy-first alternative to uploading your docs to a cloud service. Setup is rough; results are worth it.
16. NVIDIA PersonaPlex: The Voice AI That Listens and Speaks at the Same Time — If your next step is local voice AI (true full-duplex, 18x lower latency than Gemini Live), this is the architecture deep-dive.

About the Author

Kunal Ganglani is a developer and AI systems writer who tests local LLM setups, hardware configurations, and agent frameworks so you don't have to start from scratch. He writes about the infrastructure layer of AI — the hardware, tooling, and architecture decisions that determine whether a project ships or gets stuck in dependency hell. Find him at kunalganglani.com or subscribe for new guides at kunalganglani.com/subscribe.

The Local LLM Hardware Cheatsheet · 2026 Edition · kunalganglani.com Share freely. Link back. Don't sell it.